NoCDAS Documentation: A Transition to cNoC for LLM Inference

Elio Vinciguerra

May 6, 2026

Abstract

This document details the architecture and operational paradigms of NoCDAS (NoC-based Deep Neural Network Accelerator Simulator), specifically focusing on its recent extensions to support Large Language Model (LLM) inference. To overcome the memory wall bottleneck inherent in traditional Compute-at-PE architectures, NoCDAS introduces the computational Network-on-Chip (cNoC) paradigm. By outfitting routers with Multi-way Function Units (MFUs), the simulator enables in-transit computation, significantly reducing massive parameter movement across the chip. Key architectural enhancements include hardware-aware memory modeling, Rolling KV-Cache logic for infinite-length sequence generation, deterministic source routing via Serpentine Sort, and in-network online softmax reduction. This documentation provides a comprehensive overview of the system's high-level architecture, the comparative advantages of the cNoC execution mode over the baseline, and a detailed technical reference for the underlying C++ classes and data structures.

Contents

1 Introduction and Overview

1.1 What is NoCDAS

NoCDAS (NoC-based Deep Neural Network Accelerator Simulator) [1] is a cycle-accurate simulation framework designed to evaluate the performance, latency, and power characteristics of hardware accelerators for Deep Neural Networks (DNNs). While the original version focused primarily on Convolutional Neural Networks (CNNs), this modified version has been extensively re-engineered to support the unique architectural demands of Transformer-based Large Language Models (LLMs).

The core of this modification lies in the transition from a traditional *Compute-at-PE* paradigm to a hybrid architecture that incorporates computational Network-on-Chip (cNoC) [2] paradigm. This allows the simulator to model modern LLM workloads by distributing computation across the network fabric, thereby reducing the *memory wall* bottleneck associated with massive parameter movement.

1.2 Key Extensions for LLM Support

To bridge the gap between CNN acceleration and LLM workloads, the following high-level extensions were implemented:

- Advanced functional support: Integration of LLM-specific operators including Multi-Head Attention (MHA), Rotary Positional Embeddings (RoPE), RMSNorm, and the SwiGLU/GeGLU activation functions.
- The cNoC paradigm: Introduction of a cNoC mode where routers are equipped with a Multi-way Function Unit (MFU). This enables in-transit operations such as partial-sum accumulation (MatMul) and distributed Online Softmax directly within the NoC routers, further details on this architectural choice are provided in Section 8.1.
- Hardware-aware memory modeling: Detailed modeling of local SRAM constraints, including dedicated PE-level KV-Caches and Router-level weights or KV storage.
- Attention optimization: Implementation of *Attention Sinks* and *Rolling KV-Cache* logic to simulate hardware behavior during infinite-length sequence generation.
- Source routing for computation: A specialized routing protocol that ensures computation packets (type 5) follow a deterministic path through MFU-enabled routers to complete complex reductions.

1.3 Simulation Methodology

The simulator operates at a cycle-accurate granularity, modeling the hop-by-hop movement of data as *flits* through the Network-on-Chip (NoC). Every architectural component—from the Systolic Array within the MAC nodes to the Virtual

Channel (VC) allocators in the routers—respects physical hardware constraints such as buffer depths, link bandwidths, and Special Function Unit (SFU) latencies.

2 High-Level Architecture

2.1 System Topology and Interconnect

The modified NoCDAS architecture is modeled as a 2D-Mesh NoC, where each tile consists of a MAC Node (Processing Element – PE), a Network Interface (NI), and a VC Router. The system supports various grid sizes (e.g., 8x8, 12x12) and memory node configurations (e.g., `MemNode8`) to balance throughput and area.

2.2 Hardware Node Microarchitecture

Figure 1 illustrates the detailed microarchitecture of a single NoCDAS tile. It is important to note that all memory capacities and buffer depths depicted herein are purely illustrative examples; the architecture is highly scalable and these dimensions remain fully configurable parameters. The tile is composed of three heavily integrated macro-blocks:

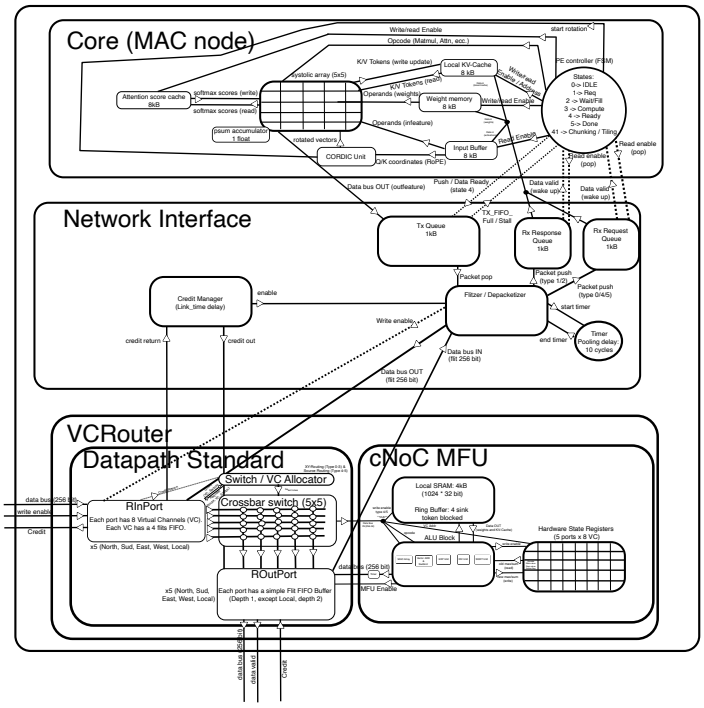

Figure 1: Detailed microarchitecture of a single NoCDAS tile.

1. Core (MAC Node): The PE is driven by an Finite State Machine (FSM) Controller and features a 5×5 Systolic Array for dense matrix multiplications. To support LLM workloads, it includes a CORDIC Unit for RoPE and an Attention Score Cache. Data locality is maintained through an Weight Memory, an Input Buffer, and a dedicated KV-Cache.

2. NI: Acting as the bridge between the compute core and the network, the NI features 1 KB (32-entry) Tx and Rx FIFOs. It contains a Flitzer/Depacketizer unit to break down messages into 256-bit flits, and a Credit Manager to handle link-time delays and backpressure mechanisms, ensuring the NoC is not overwhelmed by MAC injection rates.

3. VCRouter & cNoC MFU: The router employs a standard 5-port (North, South, East, West, Local) datapath with 8 VCs per port and a 5×5 crossbar switch. Crucially, the router is augmented with a cNoC MFU. The MFU contains a local SRAM for in-transit weights and a Ring Buffer configured to block the *Attention Sink* tokens. Further details on this architectural choice are provided in Section 6.2.3. Computation is managed via Hardware State Registers (instantiated for all 5×8 VCs) that feed into an ALU block, allowing data to be modified on the fly before being multiplexed to the output port.

2.3 The Standard Data Flow: MAC-to-MAC Loop

In a standard execution cycle (Baseline), the data movement follows a specific multi-stage pipeline across the NoC layers:

1. MAC (task generation): The controller (FSM) triggers a request for operands (weights and infeatures) based on the layer mapping.

2. NI (flitization): The NI receives the packet, breaks it into 256-bit flits (head, body, tail), and manages VC allocation and credit-based flow control.

3. Router (local to mesh): The local router accepts the flits through the `local port` and routes them into the mesh using XY-routing logic.

4. NoC traversal (router-to-router): Flits hop through intermediate routers. Each hop involves a cycle-accurate sequence of *VC selection*, *switch allocation*, and *crossbar traversal*.

5. Destination NI (depacketization): The NI at the destination node reassembles the flits into a complete message and performs backpressure checks on the Rx FIFOs.

6. Destination MAC/Memory (consumption): The data is finally consumed by the systolic array or stored in the local SRAM for computation.

2.4 The cNoC Data Flow: In-Transit Reduction

When `cNoC_MODE` is enabled, the flow is modified to perform In-Transit Computation. Instead of a pure point-to-point transfer, the packets (type 5) are intercepted by the MFU within the routers:

- MAC → NI → router: The source node injects the input vector into the NoC.
- Router-to-router (computation phase): As the flits pass through routers assigned to the task, the MFU performs local reductions (e.g., partial sum accumulation or online softmax) using the weights stored in the router's local SRAM.
- Router → terminal node: The final result is sent to the terminal node for final activation or storage.

2.5 Architectural Constraints

Every stage of this flow respects the physical hardware limits defined in `parameters.hpp`:

- Buffer depths: 32-entry FIFOs at the NI and 4-flit FIFOs per VC.
- Link latency: A deterministic `LINK_TIME` (e.g., 2 cycles) is applied to every transfer between components.

3 Baseline vs. cNoC Paradigm

The simulator allows for a direct comparison between two fundamentally different execution models: the traditional Baseline approach and the proposed cNoC architecture.

3.1 Baseline Paradigm: Data-to-Processor

In the Baseline model, the network acts as a passive transport layer. The PE are the sole entities responsible for computation:

- Mechanism: Each PE follows a *request-response* cycle. A PE identifies a task, requests weights and inputs from the Memory Controller (MC), and waits for the data to arrive before starting execution.
- Bottlenecks: This approach creates heavy congestion at the MC nodes and high latency due to the round-trip time of requests. PEs often remain idle while waiting for operands, a phenomenon known as the *memory wall*.
- Traffic pattern: High volume of unicast traffic (point-to-point) between memory nodes and various PEs across the chip.

3.2 cNoC Paradigm: In-Transit Computation

The cNoC paradigm transforms the network into an active compute fabric. Computation is performed on the fly as data flits traverse the routers:

- Mechanism: The process is split into two distinct phases:
  1. Weight distribution phase (type 4): Weights are proactively pushed from memory to the internal SRAM of specific routers along a predefined path.
  2. Computation phase (type 5): Input vectors are streamed through the network. As flits pass through each router, the MFU intercepts them, performs the operation (e.g., dot product accumulation), and updates the flit payload in-transit.
- Advantages: Data movement is significantly reduced as partial results are accumulated within the network. This minimizes the need for multiple PEs to request the same weights and allows for massive parallelism without saturating the MC nodes.
- Traffic pattern: Stream-based source routing, where a single computation packet represents an entire chain of operations.

3.3 Comparison Summary

Table 1 summarizes the fundamental differences between the Baseline and cNoC paradigms.

The most critical distinction lies in the data movement strategy: while the Baseline model pays a high penalty in terms of bandwidth and energy for transferring large weight matrices and input vectors to the PE, the cNoC model mitigates this by streaming the inputs through routers that already hold the necessary weights. Consequently, the Router Role evolves from a simple packet-forwarding switch into an active compute node equipped with a MFU.

This architectural shift drastically improves overall latency. The Baseline approach is bottlenecked by the sequential *request-transport-compute* cycle, where PEs stall waiting for memory. In contrast, cNoC pipelines the computation during the transport phase itself.

Finally, SRAM Usage is heavily optimized: instead of requiring massive local buffers at each MAC node to hold an entire layer's parameters, cNoC leverages SRAM tiling across the network fabric, utilizing the aggregated 4KB/8KB SRAM blocks of the intermediate routers.

4 Configuration and Setup

The behavior and hardware specifications of NoCDAS are centrally managed through the `parameters.hpp` header file. This file allows users to define the

Table 1: Architectural comparison between Baseline and cNoC execution modes.

Feature	Baseline (Standard)	cNoC (Proposed)
Data Movement	Operands moved to PE	Computation moved to Data
Router Role	Passive Switching	Active Processing (MFU)
Latency	Request + Transport + Compute	Pipelined In-Transit Compute
SRAM Usage	PE-centric Buffers	Distributed Router SRAM (tiling)

network topology, hardware latencies, and architectural features without modifying the core simulation logic.

4.1 The parameters.hpp Configuration Hub

The macros in this file are categorized into four main functional blocks:

4.1.1 Network Topology and Node Configuration

These macros define the physical layout of the 2D-Mesh NoC and the placement of MCs:

- Topology Selection: Macros like `MemNode8` or `MemNode32` are used to select pre-defined configurations for the grid size (`X_NUM`, `Y_NUM`) and the total number of PEs (`TOT_NUM`).
- Grid Dimensions: `PE_X_NUM` and `PE_Y_NUM` specify the number of PEs along each axis. For an 8x8 NoC, these are typically set to 8.

4.1.2 Hardware SFU Latency Parameters - Rationale and Sources

The hardware SFU latency parameters adopted in this work were selected as representative cycle-level abstractions from published hardware implementations, with priority given to throughput-oriented pipelined designs, rather than as exact replicas of a specific commercial processor. [3, 4, 5, 6]

For `MAC` and `ADD`, the choice of a one-cycle latency is consistent with fully pipelined datapaths that sustain one result per clock cycle at steady state. In particular, [3] describes a 32-bit RISC processor with an IEEE-754 floating-point unit on Zynq-7000 FPGA in which floating-point operations are executed in one clock cycle, while NoC-oriented floating-point MAC designs also report one multiply-add result per cycle at multi-GHz operating points. Similar single-cycle-throughput compute assumptions are common in CNN accelerator literature, although those designs are typically based on fixed-point or integer systolic MAC arrays rather than full floating-point units. [4, 7]

For `DIV` and `SQRT`, significantly larger latencies are consistently reported relative to addition and multiplication. Benchmark-oriented material based on published instruction tables and latency data [8] places floating-point division roughly in the 10—24 cycle range and square root in the 13—19 cycle range, depending on precision and microarchitectural details of architectures such as

Sandy Bridge. These ranges make `DIV_LATENCY = 15` and `SQRT_LATENCY = 20` reasonable mid-to-upper-range abstractions for a cycle-level model. This interpretation is further supported by FPGA implementations of low-latency reciprocal and reciprocal-square-root units, where fast approximate versions are reported at 3—12 cycles, while more accurate pipelined variants based on iterative Newton-Raphson refinement reach approximately 15—25 cycles. [9]

For the `EXP` unit, published hardware latencies span a broad range depending on precision, approximation strategy, and throughput target. Floating-point exponential operators for DSP-enabled FPGAs represent the aggressive end of the design space, with a binary32 exponential unit reported at a latency of 4 cycles. At the other end, a double-precision table-driven architecture with short Taylor expansion reports a latency of 30 clock cycles [5], while another pipelined single-precision FPGA implementation reports 19 cycles. Recent approximate architectures for exponential and hyperbolic functions further demonstrate substantial latency reductions versus prior CORDIC—and LUT—based approaches, with improvements up to 91% reported in [10] for Virtex-5/7 implementations. Within this reported range of cycles, `EXP_LATENCY = 12` can be justified as a balanced single-precision SFU estimate, positioned between very aggressive compact pipelines and more accurate but deeper implementations. [11, 12]

For `CORDIC`, the reported latency depends strongly on radix, unfolding degree, and pipeline depth, but the literature consistently places practical implementations in the range of a few to a few tens of cycles. A reduced-latency quadruple-precision floating-point arctangent based on a parallelized CORDIC scheme achieves a full result in 32 cycles at 500 MHz [6], while low-latency FPGA square-root CORDIC designs report implementation points ranging from 2 to 38 cycles [13] depending on device generation and architectural choices. Radix-4 CORDIC variants further target latency reduction relative to classical Radix-2 realizations, as validated in [14]. Consequently, `CORDIC_LATENCY = 20` lies well within the experimentally reported design space and represents a defensible intermediate value for a pipelined CORDIC-based SFU targeting 16—24-bit effective precision.

4.1.3 LLM and cNoC Functional Features

The core logic for LLM acceleration is controlled by these feature switches:

- `cNoC_MODE`: Enabling this macro activates the cNoC logic, allowing routers to perform in-transit computation.
- `ENABLE_KV_CACHE` and `KV_CACHE_SIZE`: These control the activation and capacity of the local SRAM dedicated to Key-Value tokens (e.g., 2048 floats = 8 KB).
- `ROUTER_SRAM_LIMIT`: Defines the maximum number of floats that can be stored in each router's MFU for weight distribution.

- `USE_BIAS`: Toggles the allocation and initialization of bias parameters during in-transit accumulation. When disabled (e.g., for modern bias-less models like LLaMA), the partial sum (`psum`) space is initialized to zero, saving memory bandwidth and SRAM capacity.

4.1.4 Quantization and Memory Scaling

LLM inference is notoriously memory-bound. To realistically model hardware area constraints while maintaining simulation flexibility, NoCDAS introduces the `INT8_QUANTIZATION` dynamic toggle, defined within `parameters.hpp`:

- Hardware Simulation (Fake-Quantization): The simulator employs a "Fake-Quantization" approach. The external preprocessing script quantizes the weights to INT8 to simulate the numerical clipping, but exports them back as Float32 strings. Internally, the NoCDAS C++ mathematical datapath (e.g., inside the `MAC` and `VCRouter` SFUs) operates entirely in FP32. This allows the simulator to isolate architectural performance metrics without the complexity of implementing custom INT8/FP16 mixed-precision ALUs in C++.

- SRAM Capacity Scaling: When `INT8_QUANTIZATION = 1`, the parameter `QUANT_MULTIPLIER` acts as an architectural scaler (e.g., 4x). It conceptually quadruples the effective capacity of hardware registers and local SRAM blocks (such as `KV_CACHE_SIZE` and `ROUTER_SRAM_LIMIT`) without increasing the physical silicon footprint.

- Network Traffic Reduction: The `DATA_BYTES` parameter dynamically adjusts from 4 bytes (Float32) to 1 byte (INT8). This directly affects the `Flit` payload capacity and packet alignment logic. By packing four times more parameters into a single 256-bit flit, NoC bandwidth utilization is drastically improved, significantly alleviating network congestion during the distribution of massive LLM weight matrices.

4.1.5 Physical and Frequency Constraints

These parameters define the timing relationship between components:

- `PE_NUM_OP`: Defines the number of operations per PE cycle, effectively sizing the internal Systolic Array (e.g., 25 for a 5x5 array).

- `PE_FREQ_RATIO`: Sets the frequency ratio between the NoC and the PEs. A value of 10 implies the NoC runs 10 times faster than the processing units.

- `LINK_TIME`: The deterministic latency (in cycles) for a flit to travel between two adjacent NoC nodes.

4.2 The Main Simulation Driver

The `main.cpp` file serves as the entry point and the primary orchestrator of the NoCDAS simulation. It is responsible for initializing the hardware components, loading the neural network model, and managing the main execution loop.

4.2.1 Initialization and Model Loading

Before the simulation starts, the driver performs a sequential setup of the environment:

- Model instantiation: A `Model` object is created to parse the architecture, weights, and input tensors from the filesystem.
- Evaluation modes: Depending on the macros defined (e.g., `fulleval`), the simulator loads real data or generates random tensors for architectural stress testing.
- Hardware infrastructure: The `VCNetwork` (the NoC fabric) and the `MACnet` (the compute controller) are instantiated. The `PE_NUM` is calculated based on the dimensions provided in `parameters.hpp`.

4.2.2 The Main Simulation Loop

The heart of the simulator is a monolithic loop that increments the global `cycles` counter until the model execution is complete or a maximum limit (`simulate_cycles`) is reached:

```
for(; cycles < simulate_cycles; cycles++){
    macnet->checkStatus();
    if(macnet->c_layer == macnet->n_layer) break;
    macnet->runOneStep();
    vcNetwork->runOneStep();
}
```

In each clock cycle, the following operations occur:

1. `checkStatus()`: The controller checks if the current layer has finished and prepares the mapping for the next layer.
2. `runOneStep()`: Every active component (MAC nodes, NIs, and Routers) performs its logic for the current cycle, simulating concurrent hardware behavior.

4.2.3 Correctness Verification

To ensure that the optimizations (especially in `cNoC_MODE`) do not compromise numerical integrity, `main.cpp` includes a verification block that compares the simulation output against a pre-computed `output_baseline.txt`:

- Metrics: The simulator calculates the Mean Absolute Error (MAE) and the Cosine Similarity between the baseline tensor and the simulated output.
- Success criteria: A similarity score above 98% is typically required to consider the architectural implementation successful. This is vital for validating the *In-Transit* accumulation and the *Terminal Node* activation logic.

5 Data Structures and Information Flow

The simulation depends on a clear translation between high-level model descriptions and low-level hardware tasks. This chapter describes how architectural information and data tensors are managed within the simulation environment.

5.1 The `Model` Class: Architectural Representation

The `Model` class is responsible for parsing the neural network configuration and providing the necessary metadata to the controller (`MACnet`). It manages the mapping between human-readable layer names and internal operational codes (opcodes).

5.1.1 Loading from Text Files

The simulator retrieves information from three primary text files, whose paths are defined in `parameters.hpp`:

- Model architecture: Parsed by `Model::load()`, this file defines the sequence of layers, their dimensions, and activation functions.
- Weights: Parsed by `Model::loadweight()`, it populates the `all_weight_in` deque.
- Input tensors: Parsed by `Model::loadin()`, it populates the `all_data_in` deque.

5.1.2 Opcode Mapping and Layer Types

During the execution of `Model::load()`, the simulator performs string matching to assign internal integer constants to each layer. This mapping is critical for the MFU and the PE controllers to identify the correct hardware pipeline.

The following key LLM opcodes are defined in `Model.hpp`:

- `MATMUL` (15): Standard matrix multiplication. It is the computational core of the LLM, used for linear projections (Query, Key, Value) and within Feed-Forward Networks (FFN). Formally, for an input X and a weight matrix W, it computes $Y = XW^T$.

- `LAYERNORM` (16) and `RMSNORM` (20): Normalization layers.
  - `LAYERNORM`: Applies standard normalization by centering the mean of the activations. Computed as $y = \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} \odot \gamma + \beta$.
  - `RMSNORM`: A computationally more efficient variant, adopted by models like LLaMA, which ignores the mean-centering phase and simply scales based on the root mean square: $y = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2+\epsilon}} \odot \gamma$.
- `SOFTMAX_TR` (17): Softmax for self-attention with causal mask support. During autoregressive inference, it adds an upper triangular matrix M whose non-zero elements approach $-\infty$, to zero out attention on future tokens: $\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)$.
- `ADD` (18): Element-wise addition used for residual connections (skip connections). It is essential for mitigating the vanishing gradient problem in deep models, mathematically defined as the sum of the original input and the sub-layer's output: $x_{out} = x_{in} + \mathcal{F}(x_{in})$.
- `EMBEDDING` (19): Vocabulary lookup operation. It does not require intensive computation, but rather retrieves continuous dense vectors $e_i \in \mathbb{R}^d$ from memory starting from the discrete input token ID.
- `SWIGLU` (21): Gated Linear Unit based on the swish activation function. It replaces traditional ReLU/GELU in state-of-the-art models. It modulates the flow of information by computing: $\text{SwiGLU}(x) = (xW_1 \odot \text{Swish}(xW_1)) \odot (xW_2)$.
- `ROPE` (22): Rotary Positional Embedding. It injects positional information by rotating pairs of features of the Query and Key vectors in a complex space. For a position m and an angle θ, it applies a transformation such as $q_m = qe^{im\theta}$, preserving the relative distances between tokens.
- `ATTENTION` (23): Fused hardware attention logic. It combines the entire attention computation $\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ into a single optimized kernel (like FlashAttention). This opcode drastically reduces memory bandwidth bottlenecks by executing the computation in blocks (tiling) without saving intermediate tensors.
- `GEGLU` (24): Gaussian Error Gated Linear Unit. Similar to SwiGLU, it takes a double-length input vector and splits it into a gate and up projection. The gating is modulated using the error function: $\text{GeGLU}(x) = 0.5 \cdot \text{gate} \cdot (1 + \text{erf}(\text{gate}/\sqrt{2})) \cdot \text{up}$.

5.1.3 Internal Data Structures

The extracted information is stored in high-level containers for easy access during the simulation:

- `all_layer_type`: A `deque<char>` storing the category of each layer (e.g., 'c' for Conv, 'm' for MatMul, 't' for Attention).
- `all_layer_size`: A `deque<deque<int>` containing layer-specific dimensions (channels, kernel sizes, feature lengths).
- `all_data_in` and `all_weight_in`: Nested deques storing `float` values representing the actual neural network parameters and activations.

5.2 Data Hierarchy: From Messages to Flits

To ensure efficient communication and computation, NoCDAS implements a strict data hierarchy. This structure allows the hardware to manage complex LLM tasks by breaking them down into manageable flow control units.

5.2.1 Message: The Logical Task

The `Message` struct represents the highest level of abstraction. It contains the raw data and the intent of the operation:

- Payload: A `std::vector<float>` containing input activations, weights, or partial sums. Depending on the `USE_BIAS` parameter, the starting offset of the `psum` is either initialized with the pre-loaded bias weight or strictly zeroed out for bias-less LLM architectures.
- Metadata: Includes the `compute_op` (opcode), the `sequence_id` (to track tokens), and the `routing_path` for source-routed cNoC tasks.

5.2.2 Packet: The Network Envelope

A `Packet` encapsulates a `Message` and adds network-layer information:

- Addressing: The destination is converted from a logical ID to physical coordinates (`x, y, port`) using `dest_convert()`.
- Alignment: Packet length is automatically rounded up to the nearest multiple of `FLIT_LENGTH` to ensure cycle-accurate hardware simulation.
- Virtual networks: Packets are assigned to specific Virtual Networks (`vnet`) based on their type (e.g., `vnet=1` for high-priority computation traffic).
- QoS isolation: Packets are strictly partitioned to guarantee Quality of Service (QoS) and prevent deadlocks. Standard memory read/write traffic (types 0–3) is mapped to `vnet = 0` as Unspecified Rate Service (URS). Conversely, cNoC distribution and compute traffic (types 4–5) is explicitly forced onto `vnet = 1` as Latency Critical Service (LCS). This hardware isolation ensures that intensive in-transit operations do not stall baseline memory transactions.

5.2.3 Flit: The Physical Unit

The flit is the smallest unit of transfer within the NoC. A standard packet is decomposed into three distinct types of flits: head (type 0), body (type 2), and tail (type 1). Furthermore, NoCDAS implements a hardware optimization for ultra-short messages (e.g., control signals or small scalar responses): if a packet's length is equal to a single flit, it is designated as a *Head-Tail* flit (type 10). This optimization bypasses the multi-flit pipeline, saving VC allocation cycles and reducing network congestion.

5.3 Design Choice: The `global_data_offset` Mechanism

In standard NoC simulators, routers treat flit payloads as opaque data. However, for in-transit computation, the router must know exactly which portion of the original `Message` is being carried by a specific flit.

5.3.1 Mechanism and Implementation

The `Flit` class includes a `global_data_offset` field, calculated during the *flitization* process in the NI:

$$\text{global\_data\_offset} = \text{flit\_id} \times \left(\frac{\text{FLIT\_LENGTH}}{4} \right) \tag{1}$$

This index allows the MFU in the router to map the incoming 256-bit payload directly to the correct mathematical index of the operation (e.g., identifying specific elements of a weight row).

5.3.2 Architectural Benefits

- Zero-reassembly latency: Routers can perform operations (like `MATMUL` accumulation) immediately upon flit arrival, without waiting for the entire packet to be buffered.
- Stateless processing: The router can stay *stateless* regarding the packet assembly, relying only on the offset and the opcode carried in the head flit.

6 The Network Layer

The Network Layer in NoCDAS provides the physical and logical fabric for data movement. It handles the topology, the physical links, and the orchestration of the router states across the entire chip.

6.1 The `VCNetwork` Class: Fabric Management

The `VCNetwork` class acts as the top-level container for the NoC. It is responsible for instantiating the grid of routers, connecting them via physical links, and advancing the network state in each simulation cycle.

6.1.1 Topology and Grid Construction

NoCDAS primarily implements a 2D-Mesh topology. During initialization, the `VCNetwork` constructor performs the following steps:

- Router instantiation: It creates a grid of `VCRouter` objects based on `router_num_x` (width) and `router_num_y` (height).
- Coordinate mapping: Each router is assigned a unique ID and a set of 2D coordinates (x, y) calculated as $id\_x = i/X\_NUM$ and $id\_y = i\%X\_NUM$.
- NI attachment: For each router, the network attaches the corresponding NI nodes, which serve as the entry points for the MAC compute nodes.

6.1.2 Link Connectivity

The network establishes point-to-point connections between routers. In a standard mesh, each internal router is connected to its four neighbors (North, South, East, West) and a local port for the NI. The links are modeled using the `Link` class, which encapsulates the physical latency (`LINK_TIME`) and the credit-based flow control signals required to prevent buffer overflows.

6.1.3 Simulation Orchestration

The `VCNetwork::runOneStep()` method is the engine of the network simulation. In every clock cycle, it iterates through all routers and NIs, triggering their internal logic:

- Router evolution: Calls `VCRouter::runOneStep()` to perform routing, VC allocation, and switch traversal.
- NI evolution: Calls `NI::runOneStep()` to handle flitization, packet injection, and reception.

6.1.4 Global Network Utilities

The class also provides global control functions for complex LLM workflows:

- `clearAllRouterSRAM()`: Resets the distributed memory in all routers. This is vital when transitioning between different neural network layers in `cNoC_MODE`.
- Statistics collection: Methods like `average_LCS_latency()` and `show_URS_distribution()` aggregate performance data from all NIs to report end-to-end metrics.

6.2 The `VCRouter`: The Core of cNoC

The `VCRouter` class is the most critical component of the cNoC extension. It evolves the traditional router from a passive switch into an active processing unit capable of executing complex LLM operations in-transit.

6.2.1 Routing Logic: XY vs. Source Routing

The `VCRouter::getRoute()` method manages how flits are directed through the mesh. The simulator supports two distinct routing modes depending on the packet type:

- Conventional XY routing (types 0–3): Used for standard memory requests and responses. The route is calculated purely based on the relative coordinates between the current router and the destination node (x_dest, y_dest). This ensures deadlock-free delivery for baseline traffic.
- Source routing (types 4–5): Used for cNoC distribution and computation tasks. The path is pre-calculated by the controller (`MACnet`) and stored in `message.routing_path`. The router extracts the next destination from the path vector, allowing packets to follow the specific sequence of MFU-enabled routers required for distributed reductions.

6.2.2 Design Choice: The Multi-way Function Unit

The `computeInTransit()` function is the heart of the MFU. It intercepts flits at the output port and performs mathematical operations before they are transmitted to the next hop:

- On-the-fly computation: Instead of buffering the entire packet, the MFU processes each flit using its `global_data_offset`. For a `MATMUL` operation, the MFU retrieves the local weight from the router's SRAM using the offset and performs a Multiply-Accumulate directly on the flit payload.
- State management: For multi-step operations like online softmax, the router uses `ComputeVCState` registers. These hardware registers track the `running_max` and `running_sum` for each VC, ensuring that the mathematical context is preserved as different flits of the same packet stream through the router.

6.2.3 Design Choice: Ring Buffer and Attention Sinks

To support long-sequence generation (LLM Inference) without exceeding the `ROUTER_SRAM_LIMIT`, the router implements a specialized memory management policy inspired by the StreamingLLM [15] framework.

- The Attention Sink phenomenon: Recent research has shown that LLMs tend to allocate a massive amount of attention score to the initial tokens of

a sequence (the *sinks*), regardless of their semantic relevance. Removing these tokens causes the model's performance to collapse.

- Implementation in NoCDAS: In `processDistributionPacket()`, the first N tokens (defined by `SINK_TOKENS`, typically 4) are designated as *sinks* and are permanently locked in the router's local SRAM.
- Rolling KV-Cache (Ring Buffer): For all subsequent tokens, the router employs a *Rolling KV-Cache* strategy. By treating the remaining SRAM as a circular buffer, the hardware can maintain a constant memory footprint. When the cache is full, the oldest non-sink tokens are evicted to make room for new ones.
- Theoretical basis: This approach follows the principle that for autoregressive generation, the model needs the most recent context and the initial context (sinks) to maintain numerical stability in the softmax calculation.

6.3 Input and Output Ports: Arbitration and Flow Control

The movement of flits between routers is managed by the `RInPort` and `ROutPort` classes. These components implement the low-level logic for VC allocation and switch arbitration, ensuring efficient and fair use of the physical link.

6.3.1 Virtual Channel Allocation

When a head flit arrives at an `RInPort`, the router must assign it an available VC in the downstream router. The allocation process depends on the virtual network (*vnet*) to which the packet belongs:

- URS allocation (vnet 0): Standard traffic (types 0–3) is mapped to the URS VCs. The allocator searches for an idle VC within the range `0` to `vn_num * vc_per_vn - 1`.
- LCS Allocation (vnet 1): High-priority computation traffic (types 4–5) is mapped to LCS VCs. These VCs have dedicated buffers to ensure that real-time computation flits are not blocked by standard data transfers.

6.3.2 Arbitration and Round Robin Logic

To decide which VC gets access to the crossbar switch in a given cycle, the `RInPort::getSwitch()` method implements a round robin arbitration strategy:

- Fairness: The arbiter maintains pointers (`rr_record` and `rr_priority_record`) that track the last VC served. In the next cycle, the search for a requesting VC starts from the next position, preventing any single VC from monopolizing the output port.

- Starvation avoidance: While LCS traffic generally has higher priority, a strict priority scheme could lead to *starvation* of standard traffic. NoCDAS uses a `STARVATION_LIMIT` (defined in `parameters.hpp`). If LCS flits are served for more than `STARVATION_LIMIT` consecutive cycles, the arbiter is forced to serve a pending URS flit, ensuring forward progress for all traffic classes.

6.3.3 ROutPort and Credit-Based Flow Control

The `ROutPort` acts as the final stage before the physical link. It works in tandem with the `Credit Manager` in the upstream `RInPort`:

- Backpressure: Flits are only transmitted if the downstream `RInPort` has available buffer space. This is managed through a credit system where the `ROutPort` decrements a credit counter upon transmission, and the downstream port increments it back (with a `LINK_TIME` delay) once a flit is consumed.

6.4 The Network Interface: Bridging Compute and Network

The NI acts as the essential translation layer between the parallel, task-oriented domain of the MAC nodes and the serial, flit-oriented domain of the NoC fabric. It ensures that the compute core remains agnostic of the network's internal complexities.

6.4.1 Domain Translation and Message Handling

The NI performs two primary functions to bridge these domains:

- Injection (Compute → Network): When a MAC node completes a task or requests data, it pushes a `Packet` into the NI's outbound queue. The NI then *flitizes* the packet, breaking it into 256-bit units and adding the necessary routing headers.
- Ejection (Network → Compute): Upon receiving flits from the local router, the NI's depacketizer reassembles them into a complete `Message`. Once the message is fully reconstructed, it is placed in the `packet_buffer_out` to be consumed by the MAC node.

6.4.2 Design Choice: Realistic Backpressure

To simulate a hardware-accurate implementation, the NI implements a robust backpressure mechanism. This prevents data loss and models the physical constraints of finite memory buffers.

- FIFO depth management: The NI utilizes fixed-size buffers, defined by `NI_TX_FIFO_DEPTH` and `NI_RX_FIFO_DEPTH` (typically 32 entries).

- The stalling mechanism: If the `packet_buffer_out` (the reception queue for the MAC) reaches its maximum capacity, the NI triggers a *Stop* signal. In this state, the NI refuses to absorb new flits from the network.
- Credit propagation: By stopping flit absorption, the NI ceases to return credits to the local router's `ROutPort`. This causes the router's buffers to fill up, effectively propagating the backpressure signal through the mesh and slowing down the source of the traffic.
- Architectural reasoning: This behavior is critical for modeling LLM workloads, where large bursts of attention scores or weights can easily saturate local node memories. Without this realistic backpressure, the simulation would incorrectly report zero-latency consumption, leading to non-physical performance results.

7 The Compute Layer

The Compute Layer represents the functional heart of NoCDAS, where the high-level neural network operations are translated into hardware cycles. This layer is managed by a hierarchical control structure: the global `MACnet` controller and the individual `MAC` PEs.

7.1 The `MACnet` Global Controller

The `MACnet` class acts as the centralized control unit of the simulator. Its primary responsibility is to manage the lifecycle of each layer and coordinate the transition between different execution paradigms.

7.1.1 Layer Lifecycle and Status Tracking

The controller maintains a global state machine through the `checkStatus()` method. In every cycle, it monitors the progress of the current layer:

- Completion check: It counts the number of finished tasks across all nodes. Once a layer is complete, it triggers a *global reset* of the network (e.g., clearing router SRAMs) and prepares the metadata for the next layer.
- Execution mode selection: Based on the `cNoC_MODE` macro and the layer type, `MACnet` decides whether to use the standard PE-based execution or the distributed in-transit compute mode.
- Centralized normalization pre-calculation: For layers requiring global statistical reductions across the entire feature map, such as `LAYERNORM` and `RMSNORM`, the controller performs a centralized pre-calculation of the mean, variance, or RMS values from the global `input_table`. These scalars are then packed into the payload and distributed to the PEs, eliminating the need for complex multi-pass synchronization over the NoC.

7.1.2 The Mapping Logic: Spatial Distribution

Before a layer starts, `MACnet` must decide where the computation will happen. This is handled by specialized mapping functions:

- Baseline mapping (`mapping`, `ymapping`): These functions distribute neurons/tasks across the available PEs. The goal is to maximize parallelization by assigning tasks to PEs in a balanced manner (e.g., Row-major or Column-major distribution).
- cNoC mapping (`cNoC_mapping`): When in-transit computing is selected, this function identifies a deterministic path of routers to hold the weights. It follows a Serpentine Sort (S-shaped) pattern through the mesh to minimize link contention and ensure that the reduction path is logically continuous. Further details on this architectural choice are provided in Section 8.3.

7.1.3 Traffic Injection

Once the mapping is defined, `MACnet` handles the initial data push into the NoC (`inject_cNoC_traffic`):

- Type 4 (distribution): Pushes weight parameters from the MCs to the internal SRAM of the assigned routers.
- Type 5 (computation): Injects the input activations (vectors) into the start of the cNoC path. This method is responsible for setting the `routing_path` within the message, which the routers will use for source routing.

7.2 The `MAC` Processing Element

The `MAC` class simulates the individual PEs of the accelerator. Each PE contains a systolic array controller, local SRAM buffers, and a dedicated FSM to manage its operational lifecycle.

7.2.1 The Finite State Machine

The execution of a task within a PE is governed by the `selfstatus` register, which implements the following hardware states:

- State 0 (IDLE): The PE is inactive, waiting for the global controller (`MACnet`) to assign a new layer or task.
- State 1 (REQUEST): The PE generates a request packet (type 0) to the MC to fetch necessary weights or input features.
- State 2 (WAIT/FILL): The PE stalls, waiting for the incoming data flits from the NoC. Data is stored in the 8 KB `inbuffer` or `weight` SRAM as it arrives.

- State 3 (COMPUTE): Once all operands for the current tile are available, the PE activates its systolic array logic. The base cycles spent in this state depend on `MAC_LATENCY` and the operation complexity. To ensure cycle-accurate synchronization between the faster NoC and the slower compute nodes, the FSM scales the calculated latency using the `PE_FREQ_RATIO` (e.g., `calctime = calctime * PE_FREQ_RATIO`). This translates the PE clock domain into the global NoC clock domain before setting the wake-up cycle for the next state.
- State 4 (READY): Computation is complete. The result is stored in the `outfeature` register, and the PE signals the NI to begin packet injection.

7.2.2 Design Choice: SRAM Tiling and Chunking

To overcome physical memory constraints (`MAC_WEIGHT_SRAM_LIMIT`), NoCDAS implements a tiling mechanism. This allows the hardware to process layers that are larger than the local SRAM.

- Chunk management: Each logical task is divided into N physical *chunks*. The PE uses the `current_chunk` and `total_chunks` registers to track progress.
- Partial accumulation: If an operation requires multiple passes, the PE remains in the task loop, clearing its input buffers between chunks but preserving the `psum_accumulator`. State 4 is only reached once `current_chunk == total_chunks`.
- Hardware realism: This approach accurately models the bandwidth and latency overhead of repeated memory fetches required by LLM layers.

7.2.3 Local Attention and KV-Cache Management

For LLM workloads, the MAC node includes specialized logic to handle the Key-Value (KV) cache:

- KV-Cache integration: If `ENABLE_KV_CACHE` is active, the PE utilizes a dedicated portion of its SRAM (`kv_cache`) to store tokens from previous inference steps.
- Attention score caching: To minimize NoC traffic, the PE caches the most recent attention scores (`cached_score_row`). If the same head or row is requested in subsequent cycles, the PE retrieves the data locally instead of issuing a new network request.

8 Advanced Architectural Choices: The "Why"

This section documents the high-level design decisions that define the NoCDAS architecture. Understanding these choices is critical for any developer intending

to modify the MFU or the data-path logic, as they represent carefully balanced trade-offs between silicon area and system throughput.

8.1 In-Transit Online Softmax

One of the most distinctive features of this simulator is the implementation of the online softmax algorithm directly within the NoC routers.

8.1.1 The Problem: Two-Pass Softmax and the Memory Wall

A standard softmax operation is traditionally a two-pass algorithm:

1. First pass: Iterate through the entire vector to find the maximum value (m) and compute the sum of exponentials ($d = \sum e^{x_i - m}$).
2. Second pass: Iterate again to normalize each element ($y_i = \frac{e^{x_i - m}}{d}$).

In a NoC-based accelerator, this would require storing the entire vector in a local buffer (consuming precious SRAM) or performing multiple round-trips to the MC, creating a massive bottleneck in the *memory wall*.

8.1.2 The Solution: Online Softmax Implementation

NoCDAS utilizes the online softmax approach, which allows the maximum and the denominator to be updated incrementally as each flit passes through the router. The `ComputeVCState` registers in the MFU track the `running_max` and `running_sum` for the current stream:

- When a new flit arrives, the MFU calculates the new local maximum.
- If the new maximum is greater than the `running_max`, the existing `running_sum` is rescaled ($e^{m_{old} - m_{new}}$) before adding the new exponential terms.
- This enables a single-pass reduction that happens entirely *in-flight*.

8.1.3 The Trade-off: Silicon Area vs. Bandwidth

The implementation of online softmax presents a significant architectural trade-off:

- Silicon area cost: Each router equipped with an MFU must instantiate a SFU capable of calculating exponentials (e^x). This increases the gate count and the physical area of the router.
- Bandwidth savings: By performing the reduction in-transit, NoCDAS reduces NoC traffic for Attention layers by approximately 50% compared to a two-pass Baseline implementation. It also eliminates the need for massive buffers to store intermediate softmax scores at the PE level, as the normalization context is carried by the flits themselves.

8.2 The Terminal Node Concept

In the cNoC paradigm, the simulation distinguishes between *reduction operations* and *final activations*. This leads to the terminal node concept, where complex non-linear functions are finalized at the destination node (the terminal node) rather than inside the intermediate routers.

8.2.1 Motivation: Hardware Complexity and Area Constraints

While the MFU in the router is capable of performing additions and multiplications, implementing the full logic for advanced activation functions like SwiGLU in every router would be prohibitively expensive in terms of silicon area.

- SwiGLU/GeGLU complexity: The SwiGLU activation follows the formula:
$$\text{SwiGLU}(x, W, V, b, c) = \text{Swish}_1(xW + b) \otimes (xV + c) \tag{2}$$
Similarly, GeGLU requires an error function. Both require multiple linear projections followed by complex gating mechanisms.
- Router limitation: Intermediate routers are optimized for *accumulation*. Adding the circuitry for the element-wise multiplication ($\otimes$) and the exponential/division for the swish gating in every MFU would inflate the router's power envelope and area.

8.2.2 Operational Split: In-Transit vs. Terminal

To solve this, NoCDAS splits the workload:

1. In-Transit (routers): The routers perform the heavy lifting of the linear projections (xW and xV) as the data flits move through the serpentine path. They accumulate the partial sums (`psum_accumulator`).
2. Terminal node (MC/PE): The final flit, carrying the fully reduced partial sums, reaches the terminal node. This node is equipped with a more powerful SFU that applies the complex gating and the final element-wise product to produce the output token.

8.2.3 Benefits for Scalability

This design choice allows the NoC to scale to hundreds of routers without a linear increase in the cost of complex activation logic. Only the designated terminal nodes (typically MCs or specialized PEs) need to implement the full SwiGLU/Softmax-tail hardware, while the rest of the network remains a lean, high-throughput reduction fabric.

8.3 The Deadlock Problem and Serpentine Sort

In a NoCs, a deadlock occurs when a group of packets are waiting for resources (buffers) held by each other in a circular chain, causing the entire simulation to stall indefinitely. This risk is significantly higher in cNoC mode because packets must follow a specific sequence of compute-enabled routers.

8.3.1 Why Conventional Routing is Not Enough

Standard XY-routing is inherently deadlock-free for point-to-point communication because it enforces a strict order of traversal (horizontal then vertical), preventing cycles in the channel dependency graph. However, cNoC operations (type 5) require *multi-hop reduction*, where a single packet must visit a list of specific routers in a predetermined order to accumulate results. If these paths were generated randomly or allowed arbitrary *turns*, the resulting channel dependency graph would no longer respect any global ordering of channels. As demonstrated by classical turn model literature, unrestricted turns easily allow multiple multi-hop computation streams to form cycles of VC dependencies (circular wait), inevitably leading to deadlocks [16, 17, 18].

8.3.2 The Serpentine Sort Solution

In contrast to unrestricted routing, `MACnet::cNoC_mapping()` implements a *Serpentine Sort* (also known as an S-shaped or Hamiltonian path) to map weights and determine the deterministic `routing_path`. This approach translates the theoretical requirement of an acyclic dependency graph into a practical hardware mapping strategy:

- The traversal pattern: The path starts from a memory node and traverses the first row of routers from left to right, then drops to the next row and traverses it from right to left, continuing this alternating pattern across the grid.
- Monotonic resource ordering: By following a serpentine path, the architecture enforces a strict monotonic ordering of router resources. A packet always moves forward along this predefined 1D logical array, adhering to a total order defined by the S-curve.
- Cycle elimination: Since all packets in the computation phase follow this global spatial ordering, it is mathematically impossible to form a closed loop of dependencies in the channel dependency graph. This structural guarantee ensures that the NoC can operate at near-maximum injection rates without ever reaching a deadlock state.

8.3.3 Impact on Mapping Efficiency

Beyond strict deadlock avoidance, the Serpentine Sort inherently optimizes the physical data flow of the cNoC operations:

- Spatial locality: It ensures that logically adjacent neurons (or attention heads) in a LLM layer are mapped to physically adjacent routers. This minimizes the average hop count required to pass partial sums (*psums*) between accumulation stages.
- Deterministic latency: Because the path strictly follows a non-overlapping route without adaptive detours, the path length for a given layer size becomes entirely predictable. This allows the `MACnet` global controller to accurately estimate when a layer computation will finish based on the arrival of the tail flits.

9 Complete Technical Reference: Classes and Methods

This section provides an exhaustive list of every class, method, and function implemented in the NoCDAS source code. It serves as a low-level guide for code maintenance and feature extension.

9.1 System and Compute Layer

This section describes the highest level of abstraction in the simulator. It covers the application layer, specifically the parsing of the neural network model, the global orchestration of the simulation phases, and the mathematical computations performed by the individual PEs or MAC units.

9.1.1 `Model` Class (`Model.cpp/hpp`)

The `Model` class serves as the architectural parser and data provider for the simulation. It translates high-level neural network descriptions into hardware-interpretable parameters and manages the initial state of tensors.

- `Model()`: The constructor initializes the layer count and prepares internal structures for model topology storage.
- `generateRandomFloat()`: A utility function that generates values between -0.5 and 0.5. This is critical for synthetic benchmarking when specific weight or input files are not provided.
- `load()`: The primary parser that reads the `NNmodel` configuration file. It identifies layer types (e.g., `Conv2D`, `Dense`, `Attention`, `SwiGLU`) and stores their hyperparameters (kernel size, stride, padding, activation functions) into `all_layer_size`.
- `loadin() / randomin()`: These methods populate `all_data_in`. While `loadin` reads from external files, `randomin` generates inputs based on the required dimensions for testing the data path.

- `loadweight() / randomweight()`: These methods manage the loading of model weights. `randomweight` dynamically calculates the required tensor size for each layer type, including bias terms for linear layers and dictionary sizes for embeddings.

9.1.2 `MACnet` Class (`MACnet.cpp/hpp`)

The `MACnet` class acts as the global orchestrator. It manages the lifecycle of a simulation layer, handles the spatial distribution of tasks, and simulates the MC nodes.

- `MACnet()`: Initializes the PE array. It assigns each `MAC` unit to a specific NI ID and sets up the initial input/output pointers.
- `create_input()`: Orchestrates tensor preparation for the current execution cycle. It handles complex operations like zero-padding for convolutions and the `newpooling` optimization, which fuses convolution and pooling operations into a single NoC transaction to reduce bandwidth.
- `mapping / ymapping / rmapping`: These functions implement different spatial scheduling strategies (Linear, Y-axis centric, or Randomized) to distribute neuron/token computations across the grid, avoiding collisions with reserved Memory nodes.
- `runOneStep()`: The master simulation loop. It advances the FSM of every `MAC` unit and manages the MC logic, which receives computation results and updates the `output_table`.
- `checkStatus()`: A global synchronization barrier. It verifies if all `MAC` nodes have completed their assignments, handles layer-to-layer transitions, and manages the `layer_outputs_history` to support residual connections (`ADD` layers).
- `cNoC_mapping()`: Implements the logic for cNoC. It calculates the SRAM footprint of weights and uses a `serpentine_sort` to determine the optimal physical routing path for in-transit execution.
- `inject_cNoC_traffic()`: A specialized traffic generator that creates distribution packets (type 4) for router weights and computation packets (type 5) for in-transit vector operations.

9.1.3 `MAC` Class (`MAC.cpp/hpp`)

The `MAC` class simulates a single PE. It includes internal registers, local SRAM simulation, and the computational logic for both CNN and LLM-based operations.

- `MAC()`: The constructor sets up local hardware registers, including pointers for the KV-cache, RoPE parameters, and the internal FSM state (`selfstatus`).

- `runOneStep()`: Implements the hardware FSM (states 0—5):
  - States 1-2: Handles data requests and buffering. It extracts operation-specific data (e.g., Q, K, V for attention or Filter/Input for conv).
  - State 3: The execution engine. It handles high-level LLM logic including `Softmax`, `RMSNorm`, `RoPE`, `SwiGLU`, and `GeGLU`. For `ATTENTION`, it manages a local `kv_cache`, calculates dot-product scores, and applies causal masking. It also computes hardware latency based on `PE_NUM_OP` and specific unit delays (`CORDIC`, `EXP`, `DIV`).
  - State 4: The output stage where the PE waits for the calculated clock cycles to pass before injecting the result back into the NoC.
- `inject()`: The low-level packet generator that interface with the `NI`. It encapsulates results into packets with specific QoS and routing tags.
- `sigmoid / tanh / relu`: Efficient hardware-level approximations of standard non-linear activation functions.

9.2 Network Fabric and Node Architecture

This section delves into the macro-architecture of the NoC. It details the construction of the 2D mesh topology, the instantiation of the intelligent routing nodes, and the NIs that act as critical gateways between the computational units and the communication fabric.

9.2.1 `VCNetwork` Class (`VCNetwork.cpp/hpp`)

The `VCNetwork` class is the central container for the NoC fabric, managing the topology, instantiating nodes, and collecting network-wide metrics.

- `VCNetwork()`: Constructor. It builds the 2D grid NoC by calculating XY coordinates and connecting each router to its Right and Down neighbors. It also instantiates the specified number of NIs and maps them to the local ports of their corresponding routers.
- `runOneStep()`: The global synchronization clock-level trigger. It iterates through the network array and executes the single-cycle logic for every NI and router in the NoC fabric.
- `clearAllRouterSRAM()`: Flushes the distributed weight and KV cache memories in all routers to reset the NoC state between different deep learning layers.
- `average_LCS_latency()` / `average_URS_latency()`: Metrics aggregation functions. They compute the average latency and total flit count for LCS and URS traffic patterns.

- `port_num_f(int)`: Calculates the number of active ports for a given router based on its physical location in the mesh (e.g., 3 for corners, 4 for edges, 5 for center nodes).
- `port_utilization()` / `port_utilization_innet()`: Calculates load statistics for out-ports. The former measures all ports including NIs, while the latter isolates internal NoC routing links.
- `show_LCS_distribution()` / `show_URS_distribution()`: Prints histogram data arrays tracking the end-to-end packet latency distributions and identifies worst-case delay scenarios.

9.2.2 `NRBase` Class (`NRBase.cpp/hpp`)

The `NRBase` class is a lightweight, foundational base class designed to provide polymorphism across the major architectural entities within the NoC framework.

- `NRBase()`: Default constructor establishing the base object.
- `~NRBase()`: A virtual destructor. The inclusion of a virtual destructor is a critical C++ architectural choice: it ensures safe memory cleanup for derived objects and, more importantly, enables Runtime Type Identification (RTTI). This allows components like `RInPort` to safely use `dynamic_cast` to identify whether their owning node (`router_owner`) is a `VCRouter` or an `NI` during arbitration and switching.

9.2.3 `VCRouter` Class (`VCRouter.cpp/hpp`)

The `VCRouter` is the intelligent switching node augmented with a MFU for cNoC operations.

- `VCRouter()`: Constructor. It initializes the 5-port crossbar and reserves memory for `local_weights` and `local_kv_cache` based on the defined `ROUTER_SRAM_LIMIT`. It also initializes the `ComputeVCState` matrix to track hardware state registers per VC.
- `runOneStep()`: Orchestrates the internal microarchitectural pipeline in three distinct sequential phases: `vcRequest`, `getSwitch` (arbitration), and `outPortDequeue`.
- `getRoute(Flit*)`: The routing engine. It uses strict XY routing for standard packets, but switches to Source Routing (wormhole safe) for cNoC compute packets (types 4 and 5) by tracking a pre-calculated `routing_path`.
- `vcRequest()`: Executes the VC allocation logic by polling input ports.
- `getSwitch()`: Executes the crossbar switch arbitration using a round-robin port priority mechanism (`rr_port`).

- `outPortDequeue()`: The egress logic module. It checks the `mfu_occupied_until` counter to physically stall the router based on simulated computational delays. It also triggers MFU operations by invoking `processDistributionPacket` for type 4 packets and `computeInTransit` for type 5 packets.
- `computeInTransit()`: The ALU block. It prevents redundant executions across multiple flits by tracking `computed_routers`. It performs in-flight linear operations (MatMul, Add, SwiGLU, GeGLU) and complex spatial reductions like online softmax for attention, updating `running_sum` and `running_max` to prevent mathematical overflow while saving NoC bandwidth.
- `processDistributionPacket()`: Parses type 4 packets to populate local SRAM. For Attention operations, it implements a *Rolling KV Cache* (Streaming LLM [15] concept) that locks the first `SINK_TOKENS` and circularly overwrites the remaining SRAM buffer upon capacity limits.
- `allocateSRAM(int)`: Hardware limit verification. It checks if adding data exceeds the `ROUTER_SRAM_LIMIT`, triggering a fatal hardware exception if an overflow occurs.
- `storeWeight(float)` / `storeKV(float)` / `writeKV(int, float)`: Memory management operations that write floating-point data directly to the router's local SRAM arrays.
- `clearSRAM()`: Clears all local storage vectors, resets the `kv_token_count`, and empties the list of tasks assigned to the router.

9.2.4 `NI` Class (`NI.cpp/hpp`)

The `NI` acts as the fundamental bridge translating PE transactions into NoC flits.

- `NI()`: Constructor. It initializes the TX/RX packet buffers, flit buffers, and VC arrays. It also sets up dedicated `RInPort` and `ROutPort` instances to bridge the PE with the router.
- `runOneStep()`: The main module cycle. It sequentially triggers `packetDequeue`, `vcAllocation`, `switchArbitration`, `dequeue`, and `inputCheck`.
- `packetDequeue()`: Extracts complete packets from the MAC unit's FIFO. It invokes `flitize` if a valid packet is found.
- `flitize(Packet*, int)`: Fragments a variable-length `Packet` into standard NoC flits (head, body, tail). It dynamically allocates them to specific buffer queues based on Quality of Service (QoS) tags.
- `vcAllocation()`: Iterates through idle flits and assigns VCs using either priority-based or normal allocation strategies.

- `switchArbitration()`: Handles local arbitration for egress link access. It uses starvation counters (`STARVATION_LIMIT`) and round-robin scheduling to fairly grant the switch to queued flits.
- `dequeue()`: Pushes scheduled flits from the local NI buffer into the connected router's input port.
- `inputCheck()`: Monitors incoming flits from the network. It includes a FIFO backpressure logic: if the RX queue exceeds `NI_RX_FIFO_DEPTH`, it stalls extraction, withholding credits and applying flow-control backpressure to the upstream router.

9.3 Router Microarchitecture and Interfaces

This section focuses on the internal hardware components of the network nodes. It outlines the physical and logical interfaces, including the crossbar switches, the specialized input and output ports, and the physical links that connect adjacent routers to facilitate data transmission.

9.3.1 `Port` Class (`Port.cpp/hpp`)

The `Port` class is a foundational base class that manages the VC buffer infrastructure for both input and output interfaces within the NoC nodes.

- `Port()`: The constructor initializes the port with unique identifiers and configuration parameters, including the number of virtual networks (`vn_num`) and VCs per network. It performs the critical task of instantiating the `FlitBuffer` objects, organized into two distinct groups:
  - URS Buffers: Allocated for URS traffic, occupying the lower indices of the `buffer_list`.
  - LCS Buffers: Allocated for LCS traffic, ensuring high-priority packets have dedicated physical storage.
- `~Port()`: The destructor handles the cleanup of the architectural state by iterating through the `buffer_list` and explicitly deleting each `FlitBuffer` instance to prevent memory leaks.

9.3.2 `RInPort` Class (`RInPort.cpp/hpp`)

The `RInPort` class implements the microarchitectural logic for router input ports, handling VC state machines and arbitration.

- `RInPort()`: Constructor. It initializes the port states (Idle, Routing, VC Wait, Active) and sets the `credit_delay` for all underlying flit buffers to match the physical `LINK_TIME`.

- `vc_request()`: Executes the VC allocation phase. It reads the head flit of active buffers and queries the downstream router (via `router_owner`) to secure an output VC. It separates logic for shared priority VCs, individual priority VCs, and normal URS traffic.
- `vc_allocate() / vc_allocate_normal() / vc_allocate_priority()`: These methods are invoked by upstream nodes to secure a VC in this port. They scan available VCs based on the requested QoS tier and return the VC ID, transitioning the local state to wait for routing.
- `getSwitch()`: Implements the crossbar arbitration logic. It selects a ready flit (State 3 - Active) using round-robin pointers (`rr_record`, `rr_priority_record`) and pushes it into the `ROutPort` buffer if space allows, consuming a credit. It includes a `STARVATION_LIMIT` counter to ensure high-priority traffic does not permanently block best-effort packets.

9.3.3 `ROutPort` Class (`ROutPort.cpp/hpp`)

The `ROutPort` class represents the egress physical interface for network nodes (both routers and NIs). Inheriting directly from the base `Port` class, it manages the output VC buffers and the outgoing physical wire.

- `ROutPort()`: The constructor. It delegates the instantiation of the underlying `FlitBuffer` arrays to the parent `Port` constructor, ensuring that both normal (URS) and priority (LCS) traffic queues are correctly sized based on the provided depth. It safely initializes the `out_link` pointer to `NULL` prior to the network topology binding phase.
- `out_link`: A structural member pointer referencing a `Link` object. It establishes the directed, physical outgoing path from this output port to the downstream node's corresponding input port (`RInPort`).

9.3.4 `Link` Class (`Link.cpp/hpp`)

The `Link` class provides a high-level abstraction for the physical wires and interconnects that facilitate flit and credit propagation between adjacent routers or between a router and a NI.

- `Link()`: The constructor establishes the physical binding between ports. It initializes the link by connecting it to a specific `RInPort`, which serves as the destination for flits traversing the link.
- `rInPort` / `rOutPort`: These member pointers define the source and destination of the directed interconnect, allowing the simulation to pass flit references and flow-control signals across the NoC topology.

9.4 Queuing and Memory Management

This section outlines the buffering mechanisms utilized to temporarily store data in transit. It covers the high-level packet queues managed by the NIs prior to network injection, and the low-level flit buffers (FIFOs) used by the VCs for cycle-accurate credit-based flow control.

9.4.1 `PacketBuffer` Class (`PacketBuffer.cpp/hpp`)

The `PacketBuffer` class acts as the primary staging area within the NI. It manages a FIFO queue of complete, high-level packets before they undergo the flitization process and are injected into the NoC's VC.

- `PacketBuffer()`: The constructor initializes the buffer, binding it to a specific virtual network (`vn_num`) and explicitly linking it to its parent Network Interface (`ni_owner`).
- `enqueue(Packet*)`: Appends a newly generated or received `Packet` object to the tail of the internal `packet_queue` (implemented as a `std::deque`) and increments the internal packet counter (`packet_num`).
- `dequeue()`: Extracts and returns the `Packet` at the head of the queue. It includes hardware-level assertions (`assert(packet_num!=0)`) to strictly prevent buffer underflow operations during extraction.
- `read()`: A non-destructive read operation that peeks at the front of the queue. This allows the NI's arbitration logic to inspect the next packet's metadata (e.g., scheduled output cycle, destination, QoS) without prematurely removing it from the buffer.
- `~PacketBuffer()`: The destructor ensures complete memory safety at the end of the simulation. It iterates through any remaining items in the `packet_queue` and explicitly deletes the `Packet` pointers, preventing memory leaks.

9.4.2 `FlitBuffer` Class (`FlitBuffer.cpp/hpp`)

The `FlitBuffer` class simulates the hardware FIFO queues utilized by the VC for flow control.

- `enqueue(Flit*)` / `dequeue()`: Standard FIFO operations managing the physical flit queue. `dequeue()` also registers the release of a credit in the `credit_return_queue`, delayed by simulated hardware cycles (`credit_delay`).
- `update_credits()`: A synchronous hardware tick that sweeps the `credit_return_queue`. It frees up `used_credit` slots only when the current simulation cycle surpasses the recorded delay time, accurately modeling credit-based flow control.

- `isFull() / empty()`: Capacity checks utilized by upstream nodes to determine backpressure. `isFull()` evaluates capacity based on `used_credit` against the total physical `depth`.

9.5 Data Link and Payload Layer

This section defines the actual information units that traverse the network fabric. It explains the structure of the high-level, end-to-end packets and messages generated by the computational nodes, as well as their fragmentation into granular flits that physically move through the router pipelines.

9.5.1 `Packet` and `Message` Structures (`Packet.cpp/hpp`)

The `Packet` class wraps the `Message` struct to create end-to-end network transactions. It acts as the high-level abstraction before data is fragmented into flits.

- `Message` Struct: Contains the semantic payload of the transaction. It holds standard NoC fields (Source/Destination IDs, QoS, Type) alongside specific fields for cNoC. These include the `compute_op` identifier, the `data` vector containing inputs and partial sums, the `psum_offset`, and the mathematical accumulators (`running_max`, `running_sum`) used for distributed softmax operations.

- `Packet()`: The constructor computes the exact byte `length` of the payload based on the packet `type` (0: Request, 1-3: Responses, 4: Distribution, 5: Computation). It actively calculates the overhead for floating-point tensors and ensures hardware compatibility by rounding up the final length to be a perfect multiple of `FLIT_LENGTH` (flit alignment/padding).

- `dest_convert()`: A hardware mapping helper that translates a flat destination ID into a 3D coordinate system (Router X, Router Y, and Output Port) for XY-Routing.

- `get_next_router_dest()`: A helper function for Source Routing. It reads the `routing_path` array using the `current_path_index` to dictate the deterministic path of cNoC computation packets, preventing deadlocks.

9.5.2 `Flit` Class (`Flit.cpp/hpp`)

The `Flit` (Flow control unit) is the smallest physical transfer unit in the NoC, encapsulating chunks of the parent `Packet`.

- `Flit()`: The constructor initializes routing tags (vnet, vc, type) and tracking metadata. For cNoC compute packets, it calculates the `global_data_offset` using the flit's sequence `id` to determine exactly which slice of the parent `Packet`'s floating-point array this specific flit is carrying.

- `get_data(int)` / `update_data(int, float)`: Payload accessors used by the router's MFU. They map local flit-level indices to the correct global indices within the parent packet's data array.
- `computed_routers`: A boolean tracking vector (`std::vector<bool>`) dimensioned to the total number of nodes in the network. It flags whether a specific router has already processed this flit, preventing redundant in-transit computations if the flit stalls or loops in the buffer.
- `trace_node` / `trace_time`: Diagnostic vectors used to log the physical path and cycle-accurate timestamps of the flit traversing the network.

10 Summary

The evolution of NoCDAS from a conventional CNN accelerator simulator to a cNoC-enabled framework represents a fundamental shift in hardware modeling for LLM inference. By moving computation directly into the active network fabric, the architecture effectively mitigates the severe bandwidth limitations and memory wall bottlenecks characteristic of modern LLM workloads.

As detailed in this documentation, the integration of MFUs within the routers, coupled with advanced mapping protocols like Serpentine Sort and in-transit techniques such as online softmax, allows the simulator to execute complex spatial reductions without overwhelming the local PEs. Furthermore, the meticulous cycle-accurate modeling of hardware constraints—ranging from buffer depths and realistic backpressure to KV-cache management—ensures that the simulated performance metrics remain deeply grounded in physical reality. Ultimately, NoCDAS provides a robust, highly scalable, and configurable simulation environment for exploring and validating the next generation of computational interconnects.

References

[1] W. Zhu, Y. Chen, and Z. Lu, "Nocdas: A cycle-accurate noc-based deep neural network accelerator simulator," *ACM Trans. Model. Comput. Simul.*, vol. 35, June 2025.

[2] Z. Lu and M. Liu, "Computational network-on-chip as convolution engine," in *2024 International VLSI Symposium on Technology, Systems and Applications (VLSI TSA)*, pp. 1–4, 2024.

[3] B. Özkılbaç and T. Karacalı, "Design of risc processor with ieee754 standard floating-point instruction set in fpga using vhdl for digital signal processing applications," *Erzincan University Journal of Science and Technology*, vol. 15, no. 3, pp. 699–714, 2022.

[4] S. R. Vangal, *Performance and Energy Efficient Network-on-Chip Architectures*. Dissertation, Linköping University, Linköping, Sweden, 2007.

[5] M. Wielgosz, E. Jamro, and K. Wiatr, "Highly efficient twin module structure of 64-bit exponential function implemented on sgi rasc platform," *Computing and Informatics*, vol. 28, pp. 127–137, Jan. 2012.

[6] C. He, B. Yan, S. Xu, Y. Zhang, Z. Wang, and M. Wang, "Research and hardware implementation of a reduced-latency quadruple-precision floating-point arctangent algorithm," *Electronics*, vol. 12, no. 16, 2023.

[7] K. Kiningham, "Design and analysis of a hardware cnn accelerator," 2017.

[8] A. Fog, *Instruction Tables: Lists of Instruction Latencies, Throughputs and Micro-Operation Breakdowns for Intel, AMD and VIA CPUs*. Copenhagen University College of Engineering, feb 2012. Last updated 2012-02-29. Part 4 of a five-manual series on software optimization. Available at `https://www.agner.org/optimize/`.

[9] D. Kho, M. F. Ahmad Fauzi, and S. Lim, "Hardware implementation of low-latency 32-bit floating-point reciprocal square root," *Journal of Electrical & Electronic Systems*, vol. 07, 01 2018.

[10] A. M. Dalloo, A. J. Humaidi, A. K. A. Mhdawi, and H. Al-Raweshidy, "Low-power and low-latency hardware implementation of approximate hyperbolic and exponential functions for embedded system applications," *IEEE Access*, vol. 12, pp. 24151–24163, 2024.

[11] J. S. Hegner, J. Sindholt, and A. Nannarelli, "Design of power efficient fpga based hardware accelerators for financial applications," in *NORCHIP 2012*, pp. 1–4, 2012.

[12] C. Lin, Q. Zeng, and D. Shang, "An exponential function accelerator with radix-16 algorithm for spiking neural networks," *IEICE Electronics Express*, vol. 20, no. 8, pp. 20220393–20220393, 2023.

[13] M. Węgrzyn, S. Voytusik, and N. Gavkalova, "Fpga-based low latency square root cordic algorithm," *Journal of Telecommunications and Information Technology*, vol. 99, pp. 21–29, Mar. 2025.

[14] A. Changela, Y. Kumar, M. Woźniak, J. Shafi, and M. F. Ijaz, "Radix-4 cordic algorithm based low-latency and hardware efficient vlsi architecture for nth root and nth power computations," *Scientific Reports*, vol. 13, p. 20918, Nov. 2023.

[15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," 2024.

[16] L. Schwiebert and D. Jayasimha, "A necessary and sufficient condition for deadlock-free wormhole routing," *Journal of Parallel and Distributed Computing*, vol. 32, no. 1, pp. 103–117, 1996.

[17] C.-C. Lin and F.-C. Lin, "Minimal turn restrictions for designing deadlock-free adaptive routing," in *Proceedings of 1994 6th IEEE Symposium on Parallel and Distributed Processing*, pp. 680–687, 1994.

[18] J. Wu, "A fault-tolerant and deadlock-free routing protocol in 2d meshes based on odd-even turn model," *IEEE Transactions on Computers*, vol. 52, no. 9, pp. 1154–1169, 2003.